“智学引擎”
智能学习助手系统
中期汇报

G09：陈子涵 邓佳骏 方李庭 郭云海 魏文俐

2025.12.1

目 录

01 项目阶段性成果展示

02 技术方案展示

03 组织协调沟通

04 进度管理控制

05 风险管控与方案迭代

Part 01

项目阶段性成果展示

1.1 智能问答中心：从简单检索到混合增强

01

混合检索引擎

解决的问题：解决了传统向量检索对专有名词召回率低的问题。

实现细节：手写自定义检索器，并行执行 Rank_BM25 (关键词匹配) 与 BGE-Embedding (语义匹配)，通过算法加权合并结果，实现“既懂语义，又懂术语”。

02

双重精排与上下文优化

算法集成：引入 BAAI/bge-reranker 模型，对初步召回的 Top-N 结果进行语义重打分，剔除不相关噪声。

微创切分：实施 Sentence Window (句子窗口) 策略。检索时匹配精准的单句，生成时动态替换为前后完整的上下文窗口，大幅减少了模型的断章取义和幻觉。

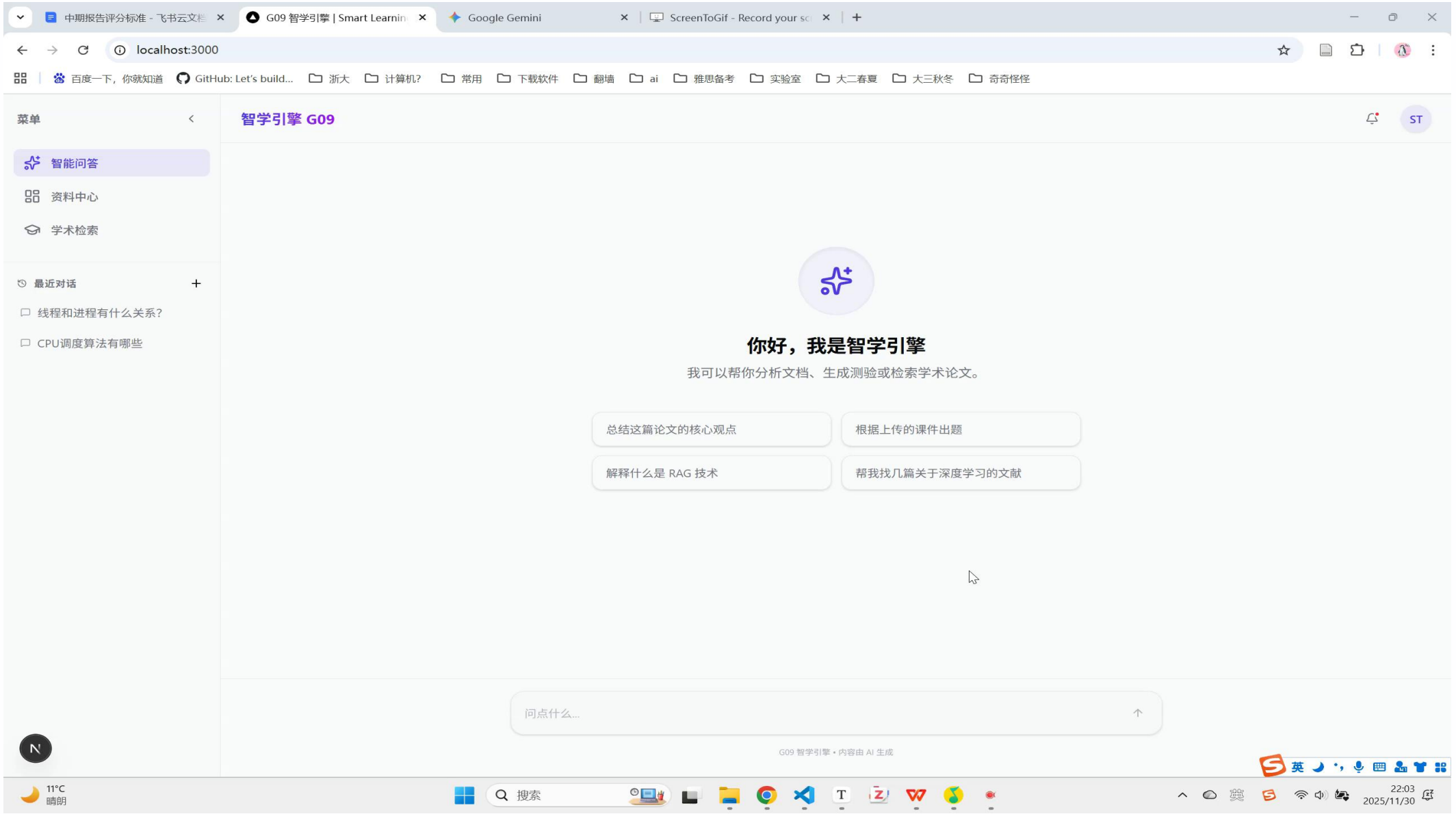
03

持久化多轮对话架构

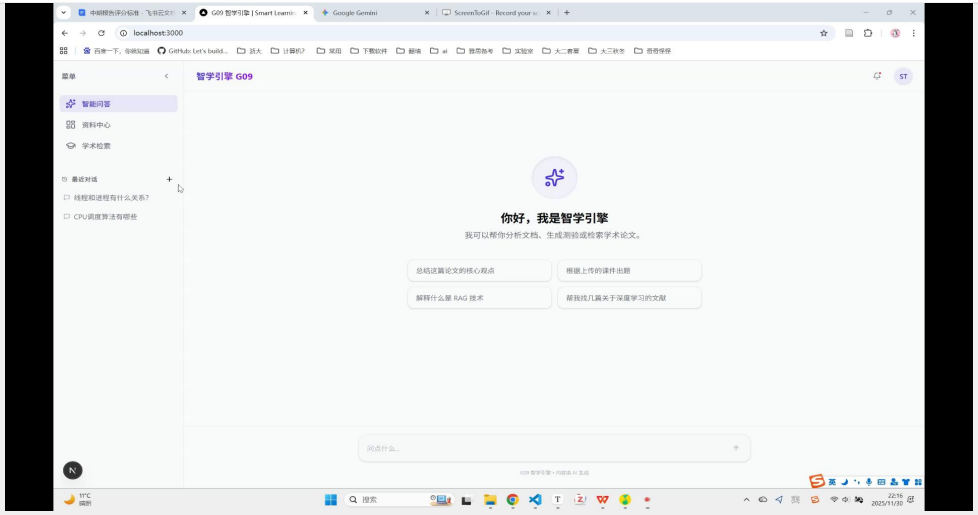
工程实现：抛弃内存存储，基于 SQLite 设计了完整的会话 (Session) 与消息 (Message) 表结构。

状态管理：实现了查询重写 (Query Rewriting) 机制，让模型能够理解“它是什么”这类指代性问题，将其还原为独立查询。

1.1 智能问答中心：从简单检索到混合增强



1.2 资料中心与AI工具集：突破限制的深度处理

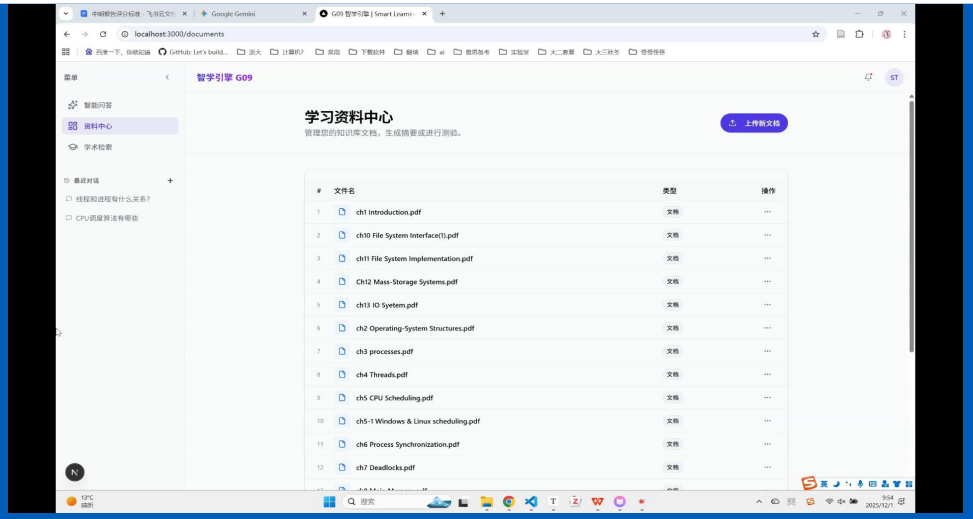


全量递归摘要

技术：摒弃简单的截断式读取，采用 TreeSummarize (树状递归) 算法。

实现逻辑：将几十页的长文档切分为多个 Chunk 并行摘要，再将中间结果递归汇总。

成果：突破了 LLM 的 Context Window 限制，实现了对全文档（而非仅前几页）的深度理解与概括，支持 Markdown 格式的渲染。



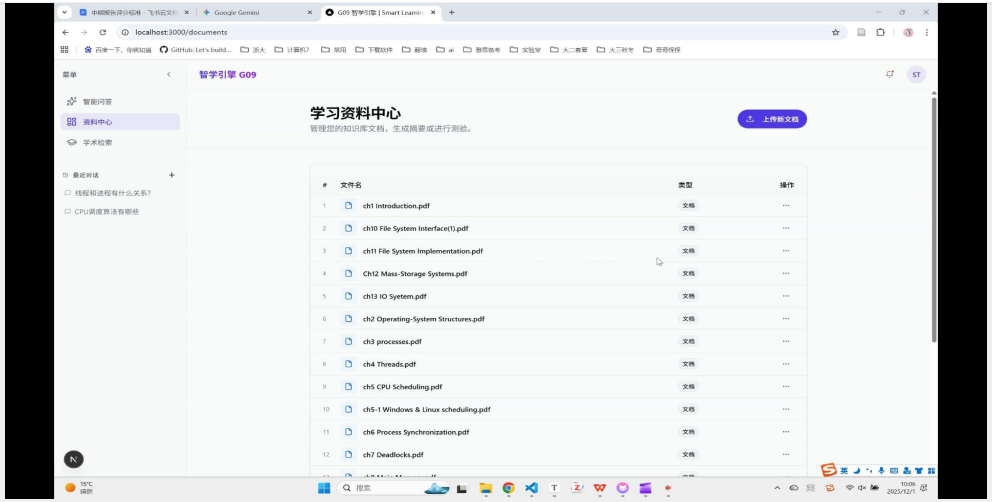
内容感知自适应测验

动态策略：后端算法实时分析文档的长度与内容特征（如是否包含公式）。

智能生成：据此动态决定题目数量与题型分布。

工程约束：通过 Prompt Engineering 强制模型输出严格的 JSON 数据结构，前端解析为带交互（解析折叠/展开）的刷题卡片。

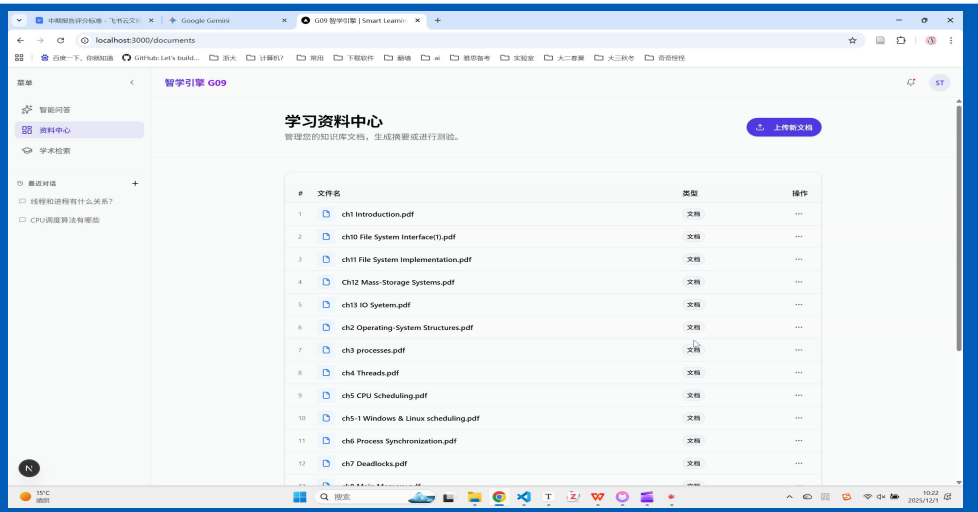
1.3 交互阅读与学术检索：沉浸式知识拓展



沉浸式划词交互

底层：改造 react-pdf 组件，监听原生 DOM window.getSelection 事件，实现毫秒级的选区捕获与 Popover 响应。

独立上下文：构建区别于全局 RAG 的临时对话通道。选中文本即刻注入独立 Prompt，实现“指哪问哪”的精准解释，且不干扰全局对话记忆。



异构数据源集成

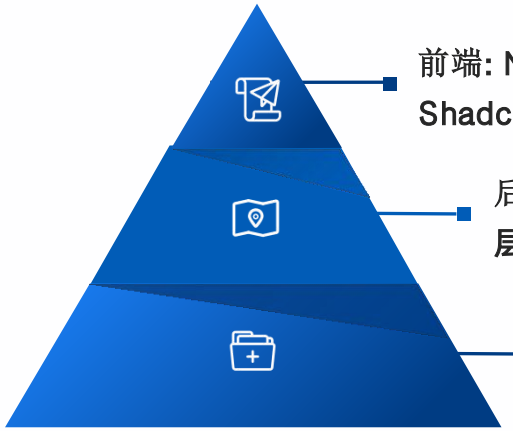
API 代理层：后端封装 Semantic Scholar 接口，统一处理跨域 (CORS) 请求与错误重试，保障服务高可用。

数据清洗：设计适配层将复杂的外部学术元数据（作者、引用、摘要）清洗为标准化的前端 Card 组件，实现本地知识与全球学术资源的连接。

Part 02

技术方案展示

2.1 总体技术架构：从MVP到模块化重构



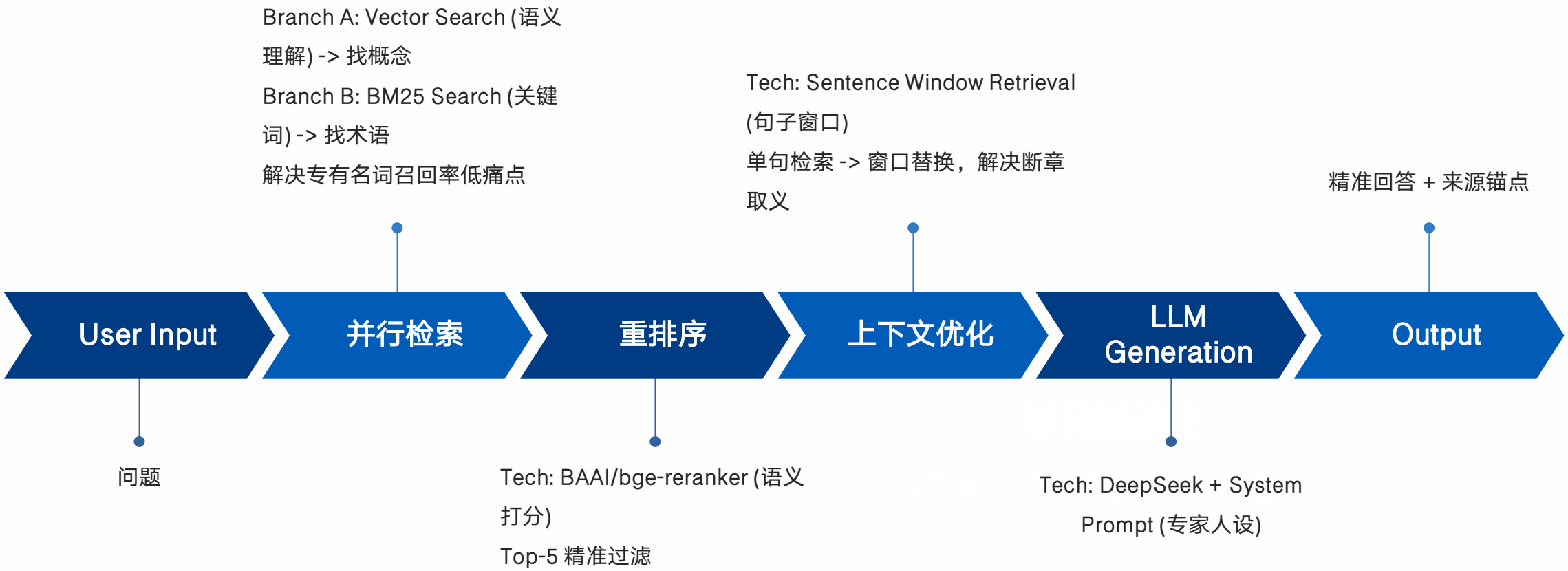
前端: Next.js 14 (App Router) + Shadcn/ui

后端: FastAPI (Router/Service 分层) + SQLite (历史记录)

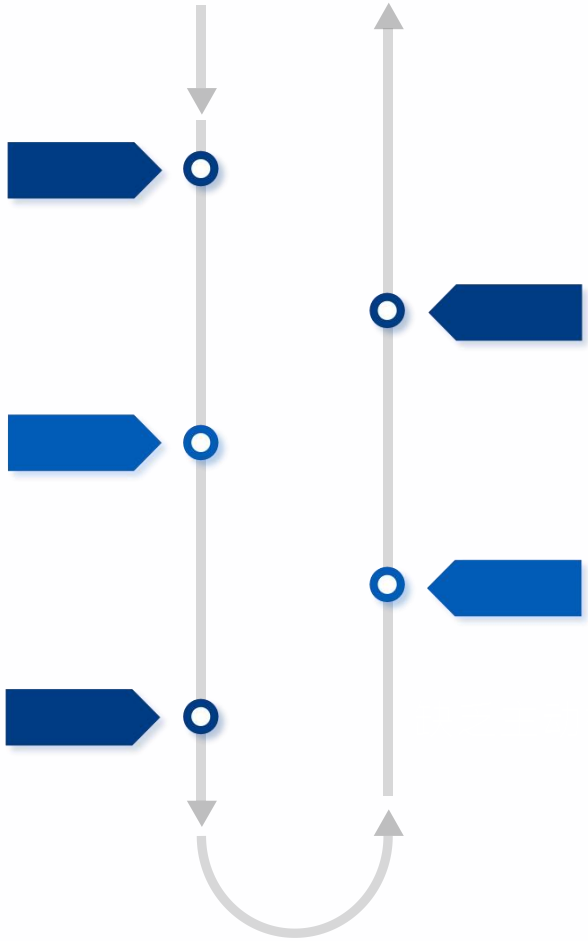
AI 核心: LlamaIndex + Hybrid Search + Reranker + DeepSeek

- 解耦：将单体脚本重构为 Router /Service /Model 三层架构，确保系统可扩展。
- 异步：全链路 async/await，支持长文档非阻塞处理。
- 规范：Pydantic 强类型约束，保障前后端数据一致性。

2.2智能问答功能闭环：解决查不准的RAG



2.3 知识处理功能闭环：突破Token限制的长文档解析



Input

几十几百页 PDF 文档

全量读取 (Full Reading)

Tech: SimpleDirectoryReader (逐页加载，不截断)

递归摘要 (Tree Summarization)

将文档切分为 Chunk A, B, C->摘要 A, B, C->汇总摘要

突破 LLM 上下文窗口限制

Output

前端渲染的 Markdown 摘要卡片 & 交互式测验题

结构化输出 (Structured Output)

Tech: Prompt Engineering (强制 JSON 格式)

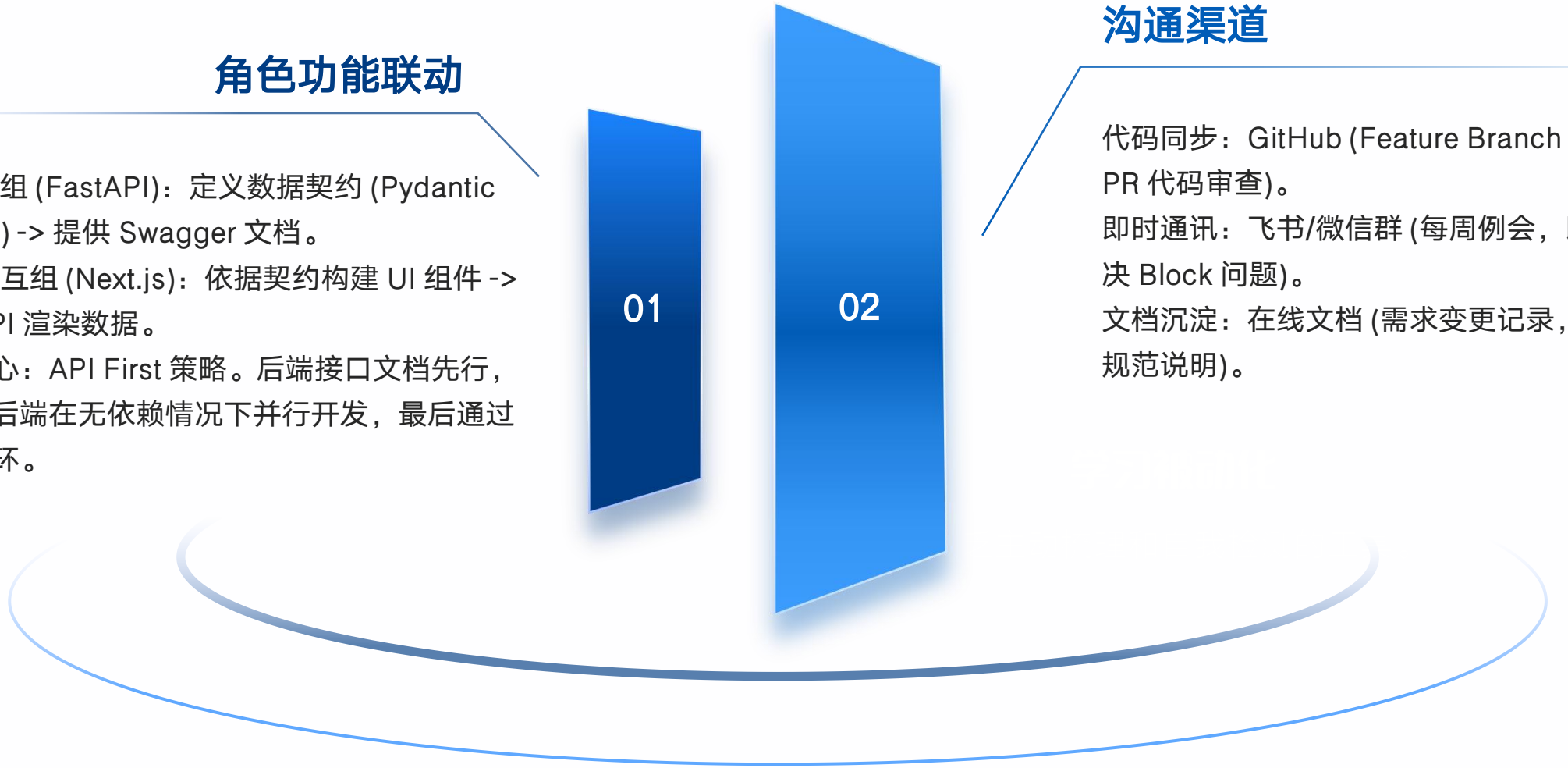
Part 03

组织协调沟通

03 通过API实现角色联动，通过Git实现代码同步

角色功能联动

后端/AI组 (FastAPI)：定义数据契约 (Pydantic Models) -> 提供 Swagger 文档。
前端/交互组 (Next.js)：依据契约构建 UI 组件 -> 调用 API 渲染数据。
联动核心：API First 策略。后端接口文档先行，确保前后端在无依赖情况下并行开发，最后通过联调闭环。



沟通渠道

代码同步：GitHub (Feature Branch 工作流，PR 代码审查)。
即时通讯：飞书/微信群 (每周例会，即时解决 Block 问题)。
文档沉淀：在线文档 (需求变更记录，接口规范说明)。

Part 04

进度管理控制

4.1 里程碑与执行追踪

10.17 - 10.24

需求与规划

顶层设计与技术选型。

☑ 修改《项目计划书》与需求池梳理。

☑ 确定全栈选型：Next.js（前端）+ FastAPI（后端）+ LlamaIndex (AI)。

10.25 - 11.07

MVP 快速构建

跑通最小可行性产品。

Sprint 1 (基础架构)：

☑ 搭建前后端分离框架，配置 CORS 与环境变量。

☑ 实现文件上传 (/api/upload) 与基础向量索引构建。

Sprint 2 (核心功能)：

☑ 跑通基础 RAG 问答链路 (Vector Only)。

☑ 集成 react-pdf 实现基础阅读器功能。

11.08 - 11.29

V2.0 深度重构

解决性能瓶颈，升级为工业级架构。

Sprint 3 (架构解耦)：

☑ 后端重构：拆解 main.py 为 Router /Service 分层架构，提升可维护性。

☑ 数据持久化：引入 SQLite，设计 Session /Message 表结构，实现多轮对话记忆。

Sprint4 (AI内核升级)：

☑ 混合检索：手写 HybridRetriever，集成 BM25 + Vector 双路召回。

☑ 精准度优化：引入 BGE-Reranker 重排序与 Sentence Window 上下文策略。

Sprint 5 (体验闭环)：

☑ 长文档攻坚：落地 TreeSummarize 递归摘要与自适应测验生成。

☑ 交互焕新：全站 UI 升级为沉浸式侧边栏布局，实现划词问答交互。

4.2 追踪机制与未来规划

进度追踪机制

1. 可视化工作量：基于 Git Commit 记录开发轨迹，坚持每周代码提交，确保每一行代码的工作量都可追溯。
2. 周迭代管理：

每周规划核心攻坚模块，每周例会同步进度。及时发现阻塞点，确保项目主线不偏航。

未来规划

1. 优化语义分块 (Semantic Chunking)

基于 Embedding 相似度突变点切分，确保每个分块都是独立的知识单元。

2. 多模态解析 (Multi-modal)

目前主要处理文本信息，忽略 PDF 中的图表、数据表格，后续考虑集成 OCR 与多模态大模型。

...

Part 05

风险管控与方案迭代

05 风险管控与方案迭代

风险类型	问题/预警 (Trigger)	迭代/处理方案 (Solution)
技术缺陷 (检索准确性)	MVP 测试中发现专有名词无法被向量检索召回，回答存在幻觉。	架构升级：引入 Hybrid Search (混合检索) 与 Reranker (重排序)，确保术语精准匹配。
可行性瓶颈 (长文档处理)	摘要功能受限于 LLM 的 Context Window，只能读取文档前几页，长文处理不可行。	算法迭代：落地 Tree Summarize (递归摘要) 策略，分块处理再递归汇总，实现全文档覆盖。
协作风险 (任务阻塞)	初期后端单文件架构导致代码冲突频发，前后端开发互相依赖，进度受阻。	管理优化：重构为 Router/Service 分层架构，确立 API 契约先行，解耦前后端开发依赖。

谢谢大家！

“智学引擎” - G09